

System Implementation and Continuous Deployment

Timothy Clarke
University of Dundee
United Kingdom
2712139@dundee.ac.uk

Video Link: <https://youtu.be/0FUa5shvAOk>

1. System Architecture & Implementation

1.1. Architectural Overview

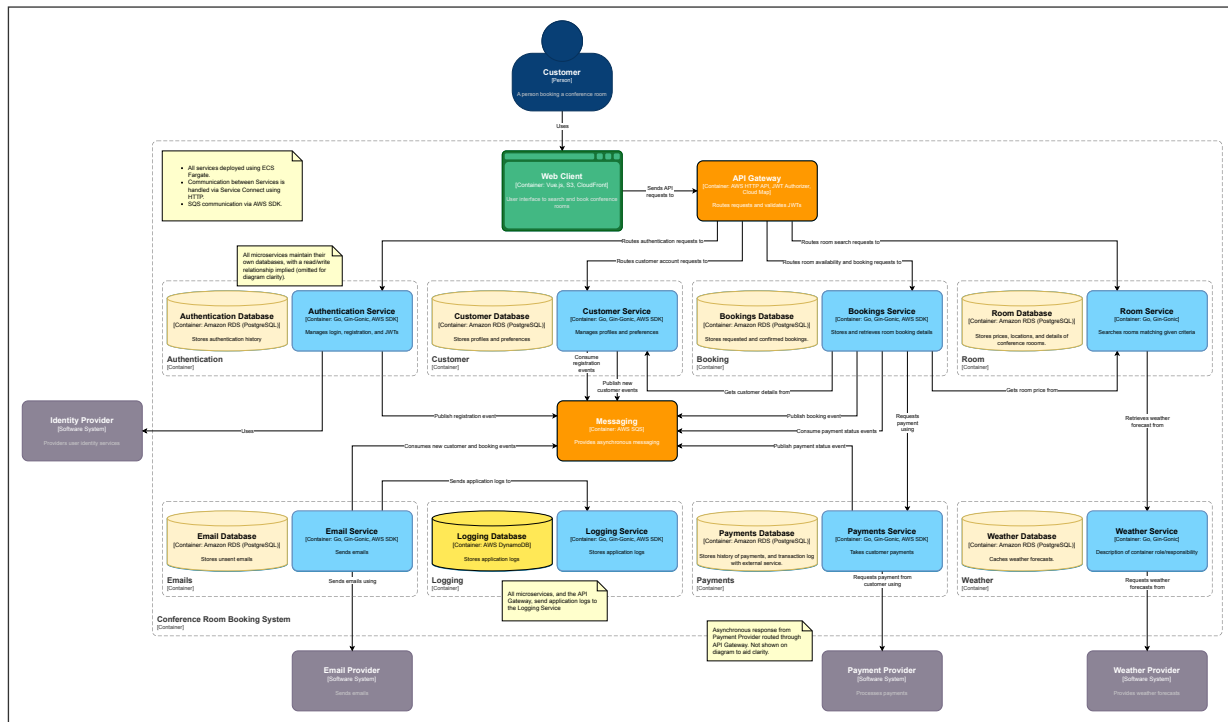


Figure 1: Updated Technology Specific C4 Level 2 Diagram

The system was implemented true to the architectural design proposed in Part 1 of the assessment (see Figure 1 above). All eight microservices were deployed as planned on the **AWS cloud platform**.

- **Microservices:** built using **Go** and the **Gin-Gonic** framework to create lightweight, high-performance **Docker** containers. These were deployed using **ECR** and **ECS**, as specified in the original design.
- **Front End:** developed using the **Vue** framework and hosted on **S3**, with delivery managed through **CloudFront**.
- **Architectural Updates:** the front-end deployment strategy was not provided in Part 1 of the assignment. Figure 1 has been updated to detail the technologies used.

1.2. Cloud Infrastructure & Orchestration

Fargate was used to create a **serverless** architecture, ensuring independent scalability and isolation for each component. Synchronous service-to-service communication is provided through **Cloud Map** and **Service Connect**, abstracting IP address management. **SQS** handles the **asynchronous messaging**. The **Vue** front end interacts with these microservices through an **HTTP API Gateway**.

Terraform orchestrates the deployment, ensuring the environment is deployed exactly as specified. By using the `for_each` meta-argument [1], the configuration minimises duplication and ensures exact service replication. A **JSON configuration** file is employed to enable easy addition of microservices and **SQS** queues (see Snippet 1 and `terraform.json`).

Five **GitHub Workflows** manage the execution of the Terraform configurations. These have native integration with AWS repositories, **preventing overhead and financial costs** of running build servers such as Jenkins. Upon pushing to the main branch, a **Blue-Green deployment strategy** [2] is triggered. This “dual-slot” architecture (using *idle* and *live* environments) is tracked via Terraform for `for_each` meta-arguments and variables, and managed through a dedicated **Switch Slots CI/CD pipeline** (see Section 2.4).

```
{
  "services": {
    "logging": {
      "queues": [],
      "db_engine": "dynamodb",
      "dynamodb_key": "ID"
    },
  },
}
```

Snippet 1: Partial JSON Configuration

1.3. NoSQL Implementation

As described in the initial design, a **DynamoDB** NoSQL database was implemented to provide a high-performance, fast-write data store for the Logger service. This allows the service to accept logs with varying metadata from different microservices without the restrictions of a rigid relational schema. **SQS queues** were used to provide a “fire-and-forget” mechanism for the client services. **DynamoDB’s Time-to-Live (TTL)** feature is used to automatically delete old records, effectively optimising storage costs (see Snippet 2 below, and db.tf).

```
resource "aws_dynamodb_table" "dynamodb" {
  # ... other configuration ...
  ttl {
    attribute_name = "ExpiresAt"
    enabled       = true
  }
}
```

Snippet 2: Partial DynamoDB TTL configuration

This logging infrastructure was also used as a mock for the external email provider. Structured email data is communicated from the Email service to the Logger service, where “sent” messages are persisted for later verification. A retrieval route was added to the Logger service, and a Vue interface implemented to display and review the stored email records.

A screenshot of the live data in the AWS Console is provided in Section 3, and Appendix A.

2. CI/CD Pipeline Plan: Tools, Processes, and Rationale

This section serves as the **CI/CD Pipeline plan**, detailing the tools, processes, and rationales used to automate the system deployment from commit to production.

The pipeline utilises **GitHub Actions** (Orchestration), **Terraform** (IaC), and **Docker** (Containerisation) to provide a cost-effective, scalable deployment environment with zero downtime, a **Green-Blue deployment strategy**, and **immediate rollback availability**.

To increase security all pipelines use GitHub’s **OIDC provider** [3] to assume the *github-terraform-role* without the need of a GitHub Secret (see Snippet 3). `terraform init` is executed at the start of every workflow job to ensure the current state of the AWS environment is known.

Automatic identification of the Green-Blue *idle* slot ensures live traffic is never interrupted while building or testing new deployments.

All the pipelines use `$GITHUB_STEP_SUMMARY` to report an outcome summary, and to advise instructions for the next step to minimise human error.

```
- name: Configure AWS Credentials
  uses: aws-actions/configure-aws-credentials@v5.1.0
  with:
    aws-region: eu-west-2
    role-to-assume: arn:aws:iam::152670839814:role/github-terraform-role
    role-session-name: Terraform
```

Snippet 3: OIDC Authentication Step

2.1. Provisioning

Creates or updates the AWS environment when manually triggered. See `create_environment.yml` for the full listing.

Action	Purpose	Rationale
Validation & Planning	Runs Terraform <code>validate</code> and <code>plan</code> commands.	Acts as a Quality Gate to catch configuration syntax errors before the <code>apply</code> stage.
Apply	Runs <code>terraform apply</code> to execute the saved plan.	Provides a reliable, repeatable, process for provisioning the environment without human intervention.
Triggers Deployment	Uses <code>gh workflow run</code> to trigger the Deploy Full Stack pipeline.	Ensures environment readiness immediately after provisioning is complete.

Table 1: Provisioning Pipeline (Create Environment)

2.2. Stack Deployment

Executed by `create_environment.yml`, or when code is pushed to the main branch of the repository. May also be triggered manually. See `deploy_stack.yml` for the full listing.

While the Blue-Green strategy provides isolation for microservices, database resources are shared between the two slots to maintain data consistency. This implementation choice introduces constraints for database schema migrations, which are managed through operational guidelines (see Section 4.2).

Action	Purpose	Rationale
Determine Slot	Queries Terraform to determine which slot is “idle”.	Implements a blue-green deployment strategy, deploying a test environment without impacting the live environment. The <i>idle</i> slot is automatically identified to reduce manual intervention and human error.
Container Deployment	Uses a <code>matrix</code> [4] to call the Deploy Container workflow for every service in parallel.	Greatly reduces the time to deploy the new microservices, and prevents version-mismatch between services. Services can be easily added to the matrix, improving scalability .
Back-End Tests	Uses <code>pytest</code> and <code>locust</code> to test the microservices against the <i>idle</i> slot.	Acts as a quality gate by halting the deployment if any functional or performance test fails.
Deploy Vue App	Only executed if the back-end tests pass. Builds the Vue app with the gateway URL synchronises with S3.	Ensures the front-end is only deployed after the back-end has been verified.
Front-End Tests	Uses <code>playwright</code> against the newly deployed Vue app.	Validates the front-end before manual switching of the <i>idle</i> slot to <i>live</i> .

Table 2: Stack Deployment Pipeline

2.3. Container Deployment

A **reusable** support workflow, executed by `deploy_stack.yml` for each microservice in parallel. See `deploy_container.yml` for the full listing.

Action	Purpose	Rationale
Provisioning	Builds the Docker image from the source folder, and deploys it to ECR with the SHA and <code>latest</code> tags.	Docker containers encapsulate a microservice’s dependencies and resources in one immutable object.
Deployment	Uses AWS CLI to force a deployment of the new image to the ECS service task	Updates the environment with the latest images, ensuring no downtime with no manual intervention, providing consistency and reducing human error.
Stability Check	Uses the AWS CLI to monitor the ECS deployment to wait until it is stabilised.	Confirms the service is fully deployed before reporting success.

Table 3: Container Deployment Pipeline

2.4. Switch Slots

A manually run workflow to switch the live environment to the currently idle environment, where new microservices should already be running. See `switch_slots.yml` for the full listing.

Action	Purpose	Rationale
Confirmation	Requires a manual keyboard input of “SWITCH” to confirm the action.	Ensures a human responsible for deployment authorises deployment. The keyboard input requirement prevents accidental deployment.
Validation	Uses AWS CLI to confirm that all services are active in the idle slot.	Prevents switching live to a partial deployment environment.
Switch	Uses <code>terraform apply</code> to switch the <i>idle</i> and <i>live</i> slots.	Provides zero downtime by changing the gateway routing to the alternative slot. Allows immediate rollback if required.
Verification	Uses <code>curl</code> to verify the gateway routing is working.	Ensures the switch is successful. If a call to a service’s health endpoint does not return a 200 HTTP status, an alert is added to the report to advise the user.

Table 4: Switch Slots Pipeline

2.5. Destroy Environment

Destroys the environment, and forces deletion of the AWS secrets. Manually run. Accidental execution is prevented by manual keyboard input. See `destroy_environment.yml` for the full listing.

Action	Purpose	Rationale
Confirmation	Requires a manual keyboard input of “DESTROY” to confirm the action.	Prevents accidental destruction of the environment.
Destroy	Uses <code>terraform destroy</code> to destroy the environment.	Ensures the environment is fully destroyed.
Delete Secrets	Use the AWS CLI to force deletion of the secrets.	Overrides the delayed deletion of secrets to allow the environment to be recreated immediately without error.

Table 5: Destroy Environment Pipeline

3. Evidence of Functionality

Figure 2 presents evidence of the deployed infrastructure and services functioning as intended. Sub-figures (a) to (d) illustrate the successful execution of the CI/CD workflow pipelines, the deployed AWS infrastructure and services, the microservices running on AWS ECS, and the DynamoDB NoSQL database functioning correctly. Together, this section provides visual confirmation that the architectural design was successfully realised in the cloud environment. See Appendix A for larger versions of these images.

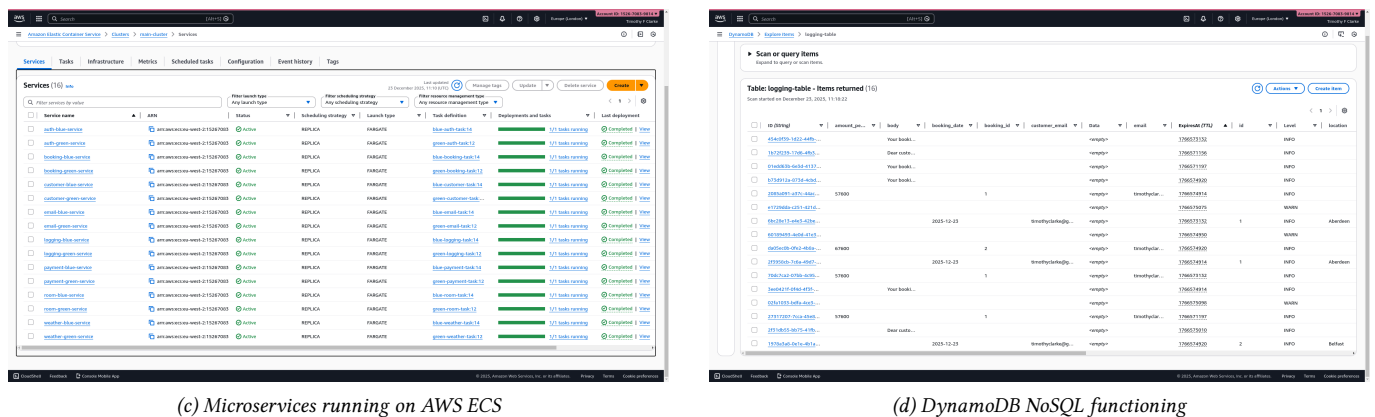
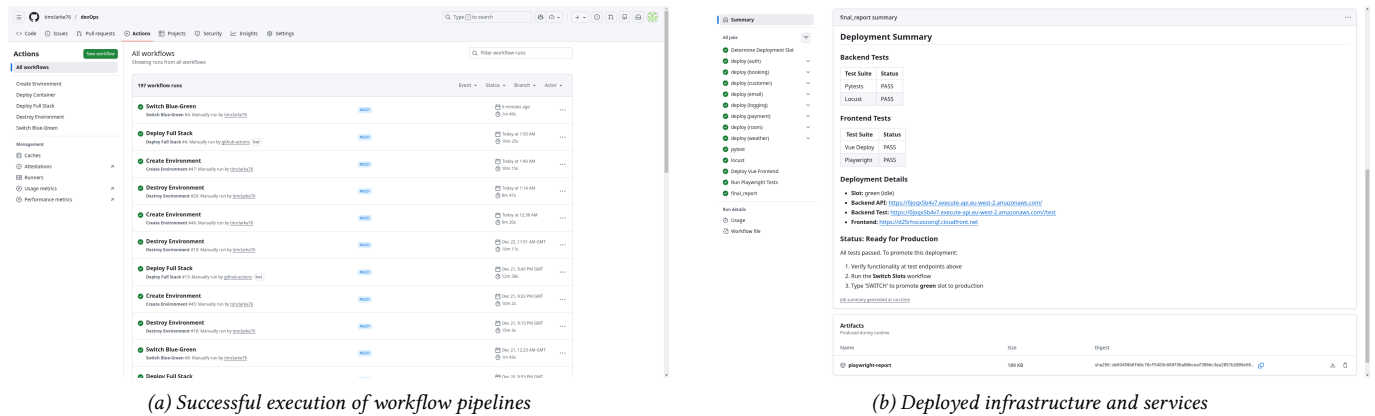


Figure 2: Evidence of deployed infrastructure and services

4. Reflection on Challenges & Solutions

4.1. Secure Networking and Subnets

Configuring the network layer for the architecture required planning to balance security and connectivity. Microservices needed to communicate with each other internally, while also accessing AWS managed services and external APIs, and allow AWS managed services to communicate back to the microservices, without providing unnecessary exposure to the public internet.

Solution: I implemented a dual-subnet architecture using Terraform's `for_each` meta-argument and `cidrsubnet` function [5] to automate the creation of resources across multiple Availability Zones (AZs).

- **Public Subnets:** Host the NAT Gateway and Internet Gateway, providing an egress path for the internal services. [6]
- **Private Subnets:** Isolate the microservices and database instances from direct public internet access.
- **Automation Logic:** As shown in Snippet 4 below, using `cidrsubnet` ensured that IP address ranges are automatically calculated, preventing errors through manual allocation.

```
resource "aws_subnet" "private" {
  for_each = toset(local.azs)
  tags = { Name = "private-${each.value}-subnet" }
  vpc_id      = aws_vpc.gateway.id
  cidr_block  = cidrsubnet(aws_vpc.gateway.cidr_block, 8,
                           length(local.azs) + index(local.azs, each.value))
  availability_zone = each.value
}
```

Snippet 4: Private Subnet Config

4.2. Blue-Green Deployment Strategy

Deploying a Blue-Green deployment strategy for the microservices architecture required careful management of the service tasks and routing to ensure the separation of *live* and *idle* environments, and allowing for switching between them by the CI/CD pipeline.

While the Blue/Green service slots are isolated, the sharing of database instances had to be considered to prevent data inconsistency.

Solution: I implemented the Blue-Green strategy by further utilising Terraform's `for_each` meta-argument to create dual instances of each microservice task definition. The *Switch Slots* CI/CD pipeline manages the routing of traffic between the *live* and *idle* environments by updating the Terraform variable that tracks the active slot.

```
variable "live_slot" {  
  type      = string  
  description = "Which slot is currently live (blue or green)"  
  default    = "blue"  
}
```

Snippet 5: Terraform live_slot variable

To manage database schema migrations without causing downtime or data inconsistency between the two environments, I chose to adopt the "Expand-Contract" pattern [7]. This approach involves ensuring that all database changes are backward-compatible, allowing both versions of the microservices to run simultaneously against the same database during the migration phase.

4.3. Microservice Deployment

Deploying eight microservices sequentially led to prolonged deployment times and increased the risk of version mismatches between services.

Solution: I used the GitHub Actions Matrix feature within `deploy_full_stack.yml`. This allowed the system to trigger the reusable `deploy_container.yml` support workflow for all services in parallel. This reduced deployment time by approximately 70% and ensured that all services were synchronised to the same version tag, preventing container incompatibility errors.

5. Conclusion

I successfully implemented an eight-service microservices architecture on AWS, fulfilling the architectural design in Part 1.

- **Go** provides fast, efficient microservices which can be deployed quickly on **AWS Fargate**.
- The combination of **Terraform** and **GitHub Actions** enables automated, repeatable deployments with minimal human error.
- The modular architecture is **scalable** and can be extended with additional services easily.
- **DynamoDB** effectively handles the logging requirements with flexible schema and automatic data expiry.
- Asynchronous communication via **SQS** provides decoupling between services.
- The **Blue-Green deployment strategy** ensures zero downtime during updates, and fast rollback if needed.

Appendix A: Evidence

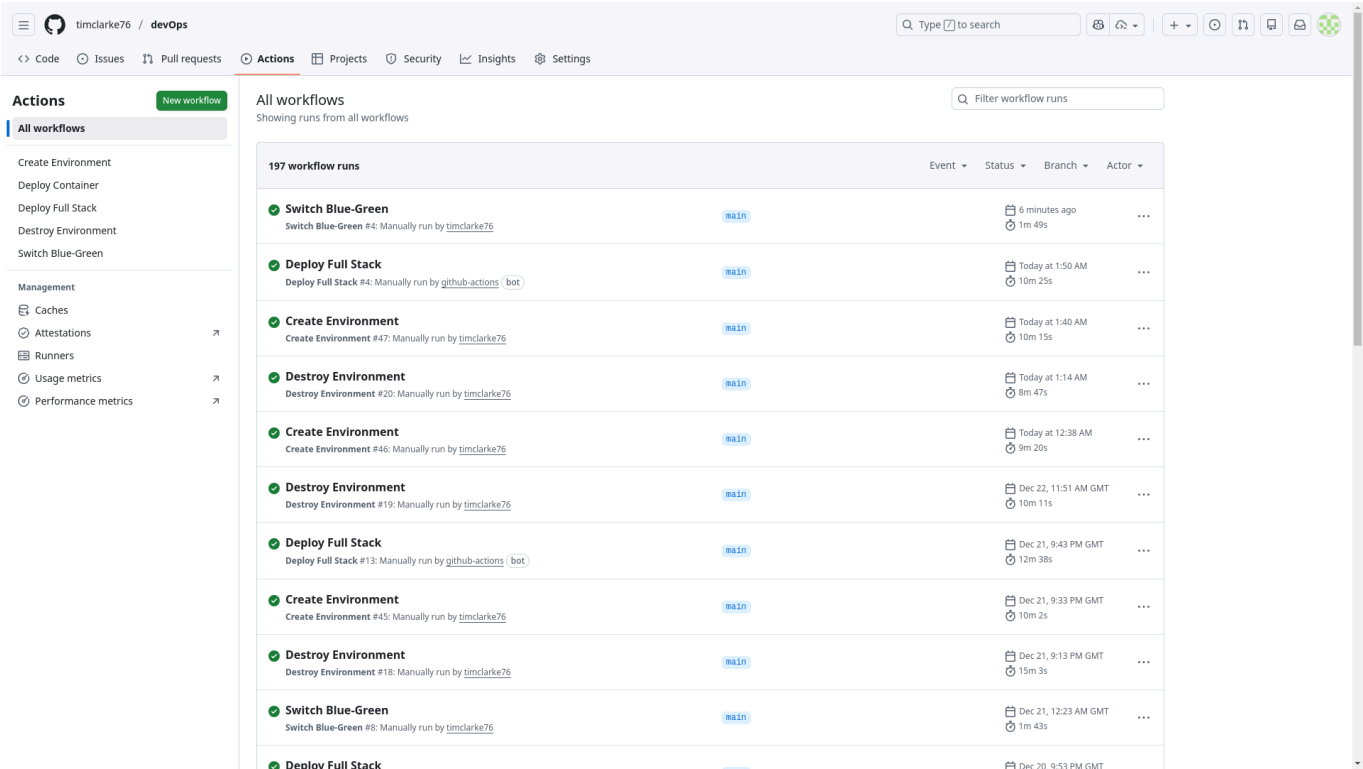


Figure 3: Successful execution of workflow pipelines

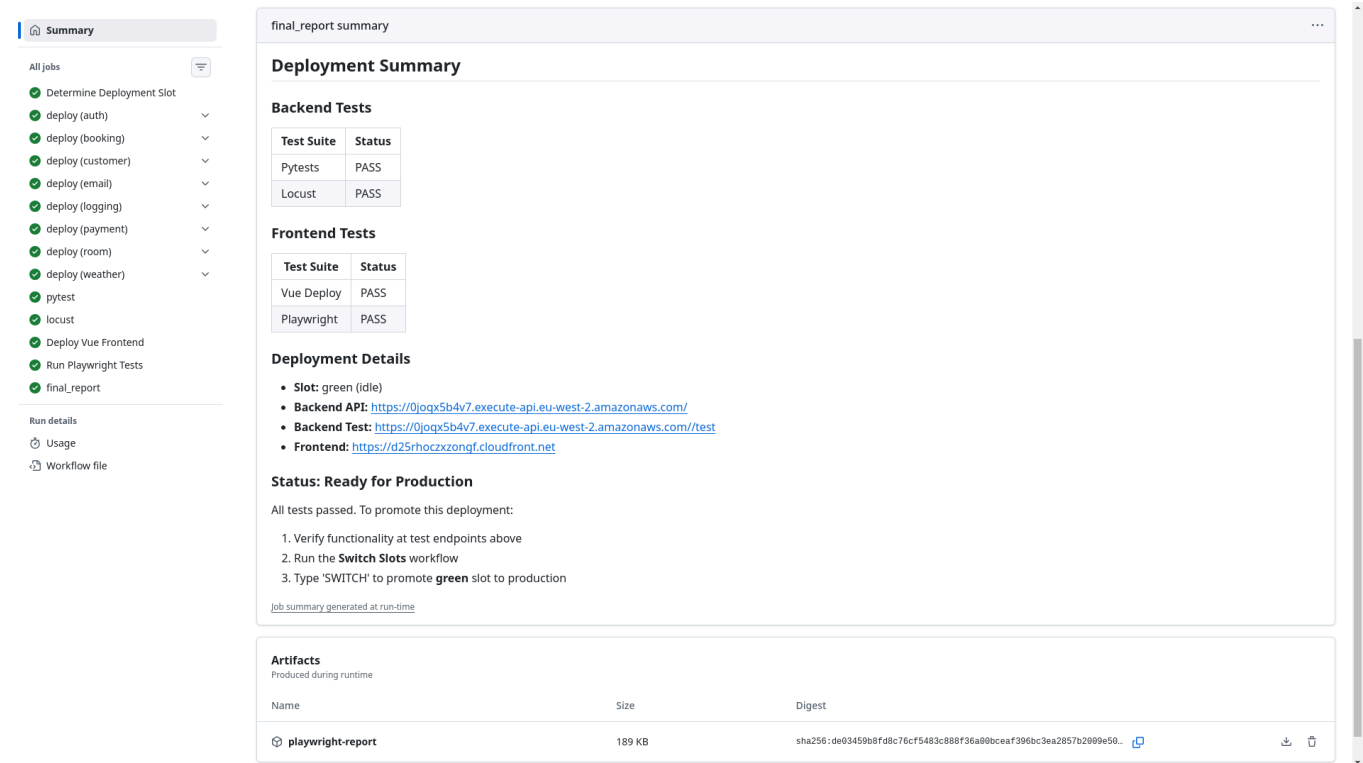


Figure 4: Deployed infrastructure and services

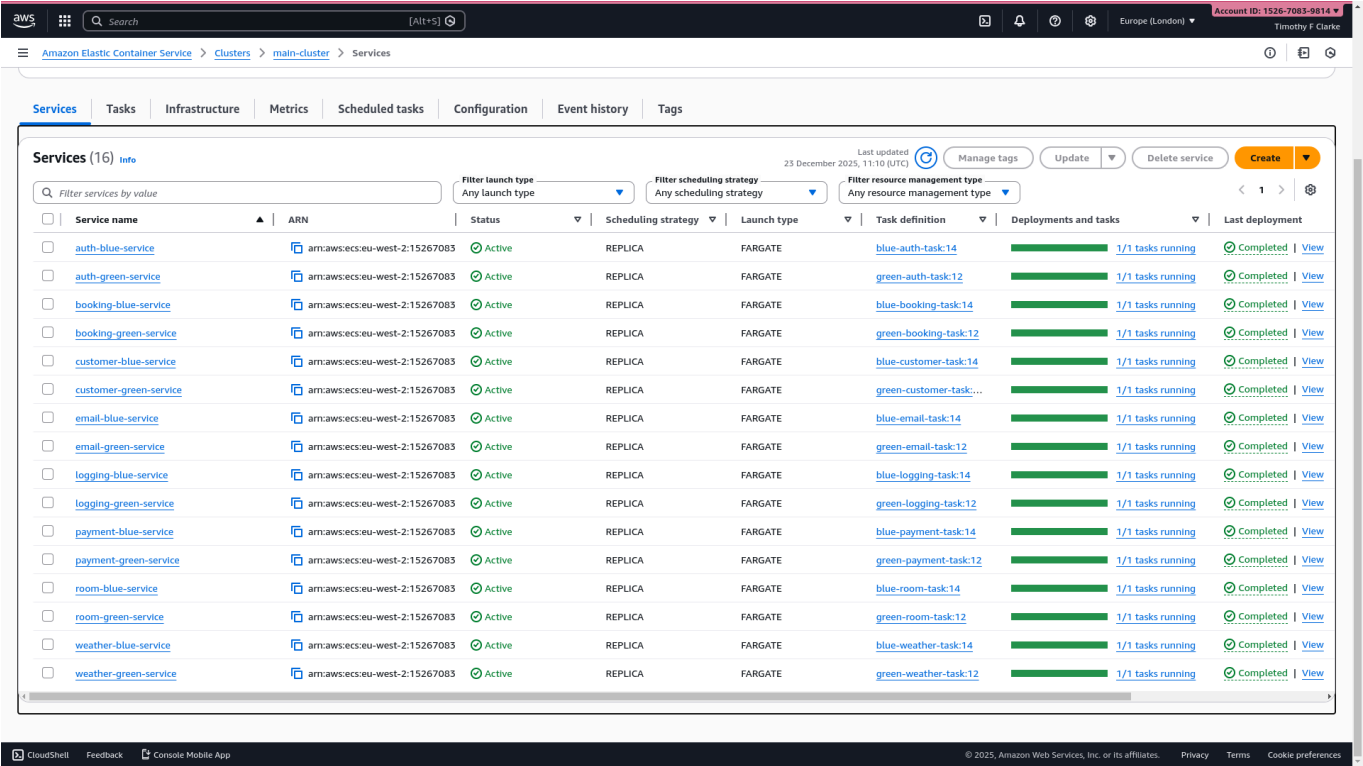


Figure 5: Microservices running on AWS ECS

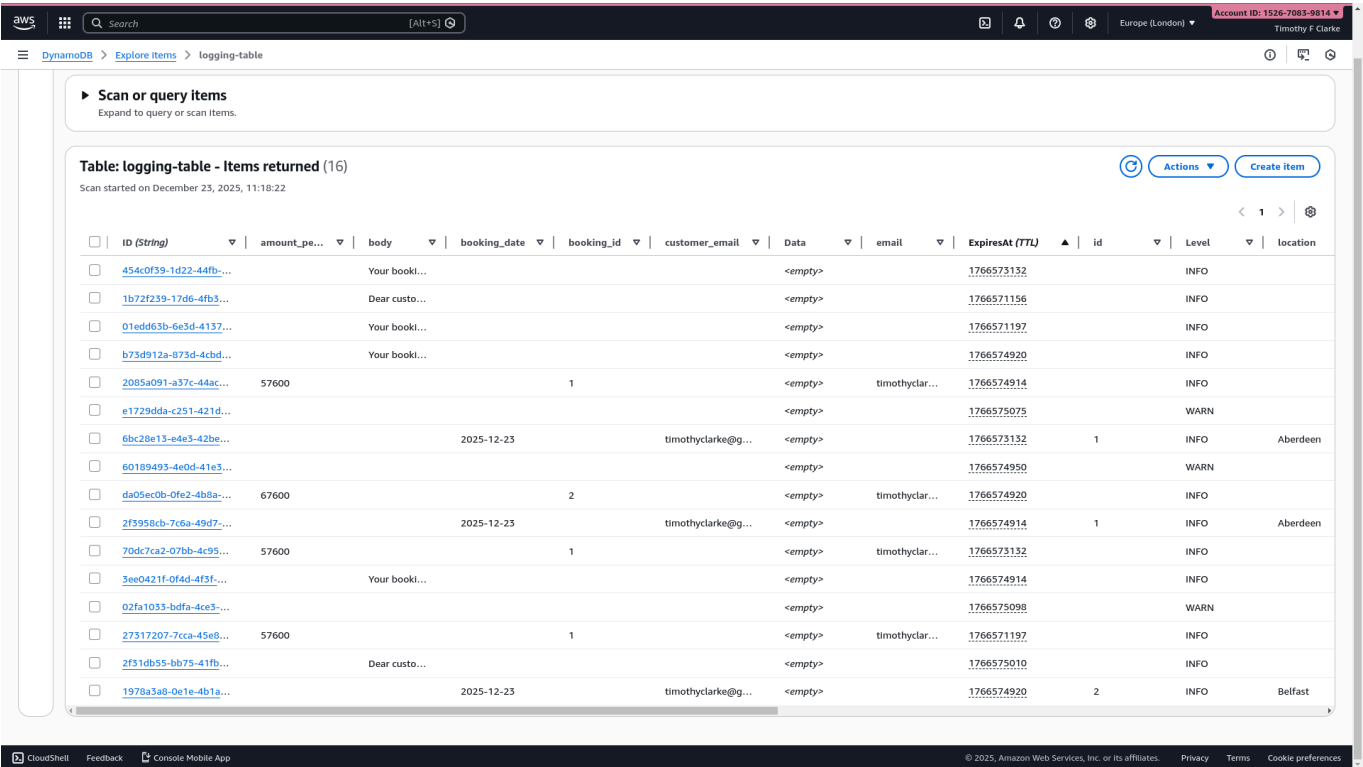


Figure 6: DynamoDB NoSQL functioning

References

- [1] “for_each meta-argument reference | Terraform | HashiCorp Developer.” Accessed: Dec. 22, 2025. [Online]. Available: https://developer.hashicorp.com/terraform/language/meta-arguments/for_each
- [2] “Blue/Green Deployments on AWS - Blue/Green Deployments on AWS.” Accessed: Nov. 16, 2025. [Online]. Available: <https://docs.aws.amazon.com/whitepapers/latest/blue-green-deployments/welcome.html>
- [3] “OpenID Connect.” Accessed: Dec. 22, 2025. [Online]. Available: <https://docs-internal.github.com/en/actions/concepts/security/openid-connect>
- [4] “Running variations of jobs in a workflow.” Accessed: Dec. 22, 2025. [Online]. Available: <https://docs-internal.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/run-job-variations>
- [5] “cidrsubnet - Functions - Configuration Language | Terraform | HashiCorp Developer.” Accessed: Dec. 22, 2025. [Online]. Available: <https://developer.hashicorp.com/terraform/language/functions/cidrsubnet>
- [6] “Example: VPC with servers in private subnets and NAT - Amazon Virtual Private Cloud.” Accessed: Dec. 22, 2025. [Online]. Available: <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-example-private-subnets-nat.html>
- [7] D. Sato, “Parallel Change.” Accessed: Dec. 22, 2025. [Online]. Available: <https://martinfowler.com/bliki/ParallelChange.html>

AI Acknowledgements

I acknowledge the use of DeepSeek to help in understanding the AWS networking requirements for this assessment. I also acknowledge the use of DeepSeek to help implement the Blue-Green deployment strategy. I acknowledge the use of Gemini to structure this assessment document, and for instructions to use the video recording software. No content generated by AI technologies has been presented as my own work.